DATABASE SCHEMA DRAFT

ELKash ERP System

Version: V1 Draft
Database: PostgreSQL (Supabase)
Architecture: Modular ERP + POS
Backend: Laravel REST API
Authentication: Laravel Sanctum

---

1. DATABASE DESIGN PRINCIPLES

Database dirancang berdasarkan prinsip:

- normalized relational structure
- modular ERP architecture
- transaction integrity
- inventory consistency
- auditability
- scalable future expansion
- immutable transactional history

Primary priorities:

1. data consistency
2. transaction reliability
3. maintainability
4. scalability

---

2. DATABASE CONVENTIONS

Naming Convention

Tables

- plural snake_case
- examples:
- users
- transaction_items
- stock_movements

Columns

- snake_case
- foreign keys:
- user_id
- product_id
- category_id

Timestamps

All transactional tables should include:

```
created_at TIMESTAMP
updated_at TIMESTAMP
```

Soft delete tables should include:

```
deleted_at TIMESTAMP NULL
```

---

3. AUTH MODULE

3.1 roles

```
CREATE TABLE roles (
    id BIGSERIAL PRIMARY KEY,
    name VARCHAR(100) UNIQUE NOT NULL,
    description TEXT NULL,

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);
```

Purpose:

- RBAC authority
- access grouping

Default Roles:

- Super Admin
- Admin

- Cashier
- Manager

3.2 permissions

```
CREATE TABLE permissions (
    id BIGSERIAL PRIMARY KEY,

    code VARCHAR(150) UNIQUE NOT NULL,
    name VARCHAR(150) NOT NULL,
    module VARCHAR(100) NOT NULL,

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);
```

Example Codes:

```
pos.transaction.create
inventory.stock.adjust
report.export
dashboard.view
```

3.3 role_permissions

```
CREATE TABLE role_permissions (
    role_id BIGINT NOT NULL,
    permission_id BIGINT NOT NULL,

    PRIMARY KEY(role_id, permission_id),

    CONSTRAINT fk_role_permissions_role
        FOREIGN KEY(role_id)
        REFERENCES roles(id),

    CONSTRAINT fk_role_permissions_permission
        FOREIGN KEY(permission_id)
        REFERENCES permissions(id)
);
```

Purpose:

- many-to-many RBAC mapping

3.4 users

```
CREATE TABLE users (
    id BIGSERIAL PRIMARY KEY,

    role_id BIGINT NOT NULL,

    name VARCHAR(150) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,

    phone VARCHAR(30) NULL,
    avatar_url TEXT NULL,

    is_active BOOLEAN DEFAULT TRUE,

    last_login_at TIMESTAMP NULL,

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW(),
    deleted_at TIMESTAMP NULL,

    CONSTRAINT fk_users_role
        FOREIGN KEY(role_id)
        REFERENCES roles(id)
);
```

Indexes:

```
CREATE INDEX idx_users_role_id
ON users(role_id);

CREATE INDEX idx_users_email
ON users(email);
```

Business Rules:

- email unique
- password hashed

- soft delete preferred

---

4. INVENTORY MODULE

4.1 categories

```
CREATE TABLE categories (
    id BIGSERIAL PRIMARY KEY,

    name VARCHAR(120) NOT NULL,
    slug VARCHAR(150) UNIQUE NOT NULL,

    description TEXT NULL,

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW(),
    deleted_at TIMESTAMP NULL
);
```

4.2 suppliers

```
CREATE TABLE suppliers (
    id BIGSERIAL PRIMARY KEY,

    name VARCHAR(150) NOT NULL,

    phone VARCHAR(50) NULL,
    email VARCHAR(150) NULL,
    address TEXT NULL,

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW()
);
```

Purpose:

- prepared for Purchasing V2
- avoid major schema refactor later

4.3 products

```sql
CREATE TABLE products (
    id BIGSERIAL PRIMARY KEY,

    category_id BIGINT NOT NULL,
    supplier_id BIGINT NULL,

    sku VARCHAR(100) UNIQUE NOT NULL,

    name VARCHAR(200) NOT NULL,
    slug VARCHAR(220) UNIQUE NOT NULL,

    description TEXT NULL,

    price NUMERIC(14,2) NOT NULL,
    cost_price NUMERIC(14,2) NULL,

    stock_quantity INTEGER NOT NULL DEFAULT 0,

    minimum_stock INTEGER DEFAULT 0,

    is_available BOOLEAN DEFAULT TRUE,
    is_favorite BOOLEAN DEFAULT FALSE,

    image_url TEXT NULL,

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW(),
    deleted_at TIMESTAMP NULL,

    CONSTRAINT fk_products_category
        FOREIGN KEY(category_id)
        REFERENCES categories(id),

    CONSTRAINT fk_products_supplier
        FOREIGN KEY(supplier_id)
        REFERENCES suppliers(id)
);
```

Indexes:

```sql
CREATE INDEX idx_products_category
ON products(category_id);
```

```
CREATE INDEX idx_products_supplier
ON products(supplier_id);

CREATE INDEX idx_products_stock
ON products(stock_quantity);

CREATE INDEX idx_products_available
ON products(is_available);
```

Business Rules:

- stock authority centralized
- soft delete only
- historical transaction preserved

4.4 stock_movements

```
CREATE TABLE stock_movements (
    id BIGSERIAL PRIMARY KEY,

    product_id BIGINT NOT NULL,
    user_id BIGINT NOT NULL,

    movement_type VARCHAR(50) NOT NULL,

    quantity INTEGER NOT NULL,

    previous_stock INTEGER NOT NULL,
    current_stock INTEGER NOT NULL,

    reference_type VARCHAR(100) NULL,
    reference_id BIGINT NULL,

    notes TEXT NULL,

    created_at TIMESTAMP DEFAULT NOW(),

    CONSTRAINT fk_stock_movements_product
        FOREIGN KEY(product_id)
        REFERENCES products(id),

    CONSTRAINT fk_stock_movements_user
        FOREIGN KEY(user_id)
```

```
        REFERENCES users(id)
);
```

Recommended ENUM Values:

```
IN
OUT
SALE
ADJUSTMENT
RETURN
```

Indexes:

```
CREATE INDEX idx_stock_movements_product
ON stock_movements(product_id);

CREATE INDEX idx_stock_movements_created_at
ON stock_movements(created_at);
```

Purpose:

- immutable inventory history
- audit trail
- inventory synchronization tracking

---

5. POS MODULE

5.1 transactions

```
CREATE TABLE transactions (
    id BIGSERIAL PRIMARY KEY,

    transaction_code VARCHAR(100) UNIQUE NOT NULL,

    cashier_id BIGINT NOT NULL,

    subtotal NUMERIC(14,2) NOT NULL,

    tax_amount NUMERIC(14,2) DEFAULT 0,
    discount_amount NUMERIC(14,2) DEFAULT 0,
```

```
    grand_total NUMERIC(14,2) NOT NULL,

    payment_status VARCHAR(50) NOT NULL,
    transaction_status VARCHAR(50) NOT NULL,

    paid_at TIMESTAMP NULL,

    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP DEFAULT NOW(),

    CONSTRAINT fk_transactions_cashier
        FOREIGN KEY(cashier_id)
        REFERENCES users(id)
);
```

Recommended ENUM Values:

```
payment_status:
- PENDING
- PAID
- FAILED
- VOID

transaction_status:
- DRAFT
- COMPLETED
- CANCELLED
```

Indexes:

```
CREATE INDEX idx_transactions_cashier
ON transactions(cashier_id);

CREATE INDEX idx_transactions_paid_at
ON transactions(paid_at);

CREATE INDEX idx_transactions_created_at
ON transactions(created_at);
```

Business Rules:

- immutable after completed
- rollback required on failure
- duplicate checkout prohibited

5.2 transaction_items

```
CREATE TABLE transaction_items (
    id BIGSERIAL PRIMARY KEY,

    transaction_id BIGINT NOT NULL,
    product_id BIGINT NOT NULL,

    product_snapshot_name VARCHAR(200) NOT NULL,
    product_snapshot_price NUMERIC(14,2) NOT NULL,

    quantity INTEGER NOT NULL,

    subtotal NUMERIC(14,2) NOT NULL,

    created_at TIMESTAMP DEFAULT NOW(),

    CONSTRAINT fk_transaction_items_transaction
        FOREIGN KEY(transaction_id)
        REFERENCES transactions(id),

    CONSTRAINT fk_transaction_items_product
        FOREIGN KEY(product_id)
        REFERENCES products(id)
);
```

Purpose:

- preserve historical pricing
- immutable invoice detail
- prevent product update corruption

5.3 payments

```
CREATE TABLE payments (
    id BIGSERIAL PRIMARY KEY,

    transaction_id BIGINT NOT NULL,

    payment_method VARCHAR(50) NOT NULL,
```

```
    paid_amount NUMERIC(14,2) NOT NULL,

    change_amount NUMERIC(14,2) DEFAULT 0,

    payment_reference VARCHAR(255) NULL,

    payment_status VARCHAR(50) NOT NULL,

    paid_at TIMESTAMP NULL,

    created_at TIMESTAMP DEFAULT NOW(),

    CONSTRAINT fk_payments_transaction
        FOREIGN KEY(transaction_id)
        REFERENCES transactions(id)
);
```

Recommended ENUM Values:

```
payment_method:
- CASH
- QRIS
- DEBIT
- CREDIT_CARD
- E_WALLET
```

Indexes:

```
CREATE INDEX idx_payments_transaction
ON payments(transaction_id);

CREATE INDEX idx_payments_method
ON payments(payment_method);
```

---

6. REPORTING & AUDIT MODULE

6.1 audit_logs

```
CREATE TABLE audit_logs (
    id BIGSERIAL PRIMARY KEY,
```

```
    user_id BIGINT NOT NULL,

    action VARCHAR(150) NOT NULL,
    module VARCHAR(100) NOT NULL,

    entity_type VARCHAR(100) NOT NULL,
    entity_id BIGINT NOT NULL,

    old_values JSONB NULL,
    new_values JSONB NULL,

    ip_address VARCHAR(100) NULL,
    user_agent TEXT NULL,

    created_at TIMESTAMP DEFAULT NOW(),

    CONSTRAINT fk_audit_logs_user
        FOREIGN KEY(user_id)
        REFERENCES users(id)
);
```

Indexes:

```
CREATE INDEX idx_audit_logs_user
ON audit_logs(user_id);

CREATE INDEX idx_audit_logs_module
ON audit_logs(module);

CREATE INDEX idx_audit_logs_created_at
ON audit_logs(created_at);
```

Purpose:

- operational tracing
- immutable audit history
- security monitoring

6.2 failed_jobs

Laravel Queue support.

```
CREATE TABLE failed_jobs (
    id BIGSERIAL PRIMARY KEY,

    uuid VARCHAR(255) UNIQUE NOT NULL,

    connection TEXT NOT NULL,
    queue TEXT NOT NULL,

    payload TEXT NOT NULL,
    exception TEXT NOT NULL,

    failed_at TIMESTAMP DEFAULT NOW()
);
```

Purpose:

- future queue compatibility
- export jobs
- invoice generation jobs
- notification jobs

---

7. FUTURE SCHEMA PREPARATION (V2+)

Prepared but optional for V1.

Potential Future Tables:

```
purchase_requests
purchase_orders
purchase_order_items

sales_orders

inventory_adjustments

journals
journal_entries
ledgers

notifications

report_exports
```

Reason:

- avoid future architecture rewrite
- maintain ERP scalability

---

8. DATABASE RELATIONSHIP SUMMARY

AUTH

```
roles
 └── users
      └── audit_logs
```

INVENTORY

```
categories
 └── products
      ├── stock_movements
      └── transaction_items

suppliers
 └── products
```

POS

```
transactions
 ├── transaction_items
 └── payments
```

USER RELATIONS

```
users
 ├── transactions
 ├── stock_movements
 └── audit_logs
```

---

9. TRANSACTION INTEGRITY RULES

Critical operations must use DB transaction:

- checkout process
- payment persistence
- inventory deduction
- stock rollback
- transaction cancellation

Mandatory Rules:

- failed payment must rollback inventory
- partial persistence prohibited
- duplicate checkout prevented
- completed transactions immutable

Example:

```
DB::transaction(function () {
   // create transaction
   // create payment
   // deduct stock
   // create stock movement
});
```

---

10. INDEXING STRATEGY

Priority indexes:

```
users.email
products.sku
products.stock_quantity
transactions.transaction_code
transactions.created_at
payments.transaction_id
stock_movements.product_id
audit_logs.created_at
```

Purpose:

- faster POS lookup

- reporting optimization
- dashboard aggregation
- inventory filtering

---

11. SOFT DELETE POLICY

Soft delete enabled for:

- users
- categories
- products

Reason:

- preserve historical integrity
- maintain reporting consistency
- prevent broken transaction references

---

12. DATABASE ENGINEERING PRINCIPLES

Database prioritizes:

- consistency over extreme speed
- transactional reliability
- maintainability
- realistic ERP workflow

Avoid:

- premature optimization
- unnecessary denormalization
- microservice-oriented schema split
- overengineered event sourcing

Primary focus:

stable business process synchronization between POS, inventory, reporting, and authentication modules.